



```
In [1]: # =====
# 1. Import Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [2]: # =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())
```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

```
In [3]: # =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
```

```
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [4]: # =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()
```

C:\Users\HARSHAL CHELIKANI\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py:106: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 128)	
dense_1 (Dense)	(None, 64)	
dense_2 (Dense)	(None, 32)	
dense_3 (Dense)	(None, 1)	

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

```
In [6]: # =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
```

```
        verbose=1
    )

    # =====
    # 8. Evaluate Model
    # =====
    loss, mae = model.evaluate(X_test, y_test)
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    print("\nTest MAE:", mae)
    print("RMSE:", rmse)
    print("R2 Score:", r2)

    print("Actual vs Predicted")
    plt.show()
```

Epoch 1/100
465/465  9s 10ms/step - loss: 0.7623 - mae: 0.5704 - val_loss: 0.4325 - val_mae: 0.4682

Epoch 2/100
465/465  4s 9ms/step - loss: 0.3648 - mae: 0.4291 - val_loss: 0.3940 - val_mae: 0.4345

Epoch 3/100
465/465  4s 9ms/step - loss: 0.3408 - mae: 0.4104 - val_loss: 0.3551 - val_mae: 0.4281

Epoch 4/100
465/465  3s 5ms/step - loss: 0.3235 - mae: 0.3981 - val_loss: 0.3531 - val_mae: 0.4079

Epoch 5/100
465/465  3s 7ms/step - loss: 0.3120 - mae: 0.3900 - val_loss: 0.3506 - val_mae: 0.4101

Epoch 6/100
465/465  5s 10ms/step - loss: 0.3001 - mae: 0.3794 - val_loss: 0.3346 - val_mae: 0.4068

Epoch 7/100
465/465  5s 10ms/step - loss: 0.3085 - mae: 0.3782 - val_loss: 0.3238 - val_mae: 0.4026

Epoch 8/100
465/465  4s 9ms/step - loss: 0.2938 - mae: 0.3707 - val_loss: 0.3254 - val_mae: 0.3886

Epoch 9/100
465/465  4s 7ms/step - loss: 0.2909 - mae: 0.3701 - val_loss: 0.3299 - val_mae: 0.4029

Epoch 10/100
465/465  4s 9ms/step - loss: 0.2818 - mae: 0.3654 - val_loss: 0.3077 - val_mae: 0.3798

Epoch 11/100
465/465  4s 8ms/step - loss: 0.2772 - mae: 0.3610 - val_loss: 0.3030 - val_mae: 0.3737

Epoch 12/100
465/465  4s 8ms/step - loss: 0.2744 - mae: 0.3602 - val_loss: 0.3113 - val_mae: 0.3714

Epoch 13/100
465/465  3s 7ms/step - loss: 0.2721 - mae: 0.3562 - val_loss: 0.3032 - val_mae: 0.3817

Epoch 14/100
465/465  3s 7ms/step - loss: 0.2686 - mae: 0.3543 - val_loss: 0.3031 - val_mae: 0.3846

Epoch 15/100
465/465  4s 7ms/step - loss: 0.2685 - mae: 0.3534 - val_loss: 0.3145 - val_mae: 0.3820

Epoch 16/100
465/465  3s 7ms/step - loss: 0.2624 - mae: 0.3506 - val_loss: 0.2906 - val_mae: 0.3709

Epoch 17/100
465/465  3s 6ms/step - loss: 0.2648 - mae: 0.3512 - val_loss: 0.3154 - val_mae: 0.3903

Epoch 18/100
465/465  4s 8ms/step - loss: 0.2630 - mae: 0.3501 - val_loss: 0.3023 - val_mae: 0.3827

Epoch 19/100
465/465  4s 8ms/step - loss: 0.2583 - mae: 0.3474 - val_loss: 0.3026 - val_mae: 0.3888
Epoch 20/100
465/465  3s 7ms/step - loss: 0.2553 - mae: 0.3435 - val_loss: 0.3031 - val_mae: 0.3900
Epoch 21/100
465/465  3s 7ms/step - loss: 0.2528 - mae: 0.3448 - val_loss: 0.2876 - val_mae: 0.3678
Epoch 22/100
465/465  4s 7ms/step - loss: 0.2579 - mae: 0.3433 - val_loss: 0.2841 - val_mae: 0.3551
Epoch 23/100
465/465  4s 8ms/step - loss: 0.2469 - mae: 0.3388 - val_loss: 0.2848 - val_mae: 0.3602
Epoch 24/100
465/465  3s 7ms/step - loss: 0.2455 - mae: 0.3389 - val_loss: 0.3062 - val_mae: 0.3700
Epoch 25/100
465/465  3s 7ms/step - loss: 0.2478 - mae: 0.3388 - val_loss: 0.2814 - val_mae: 0.3590
Epoch 26/100
465/465  4s 8ms/step - loss: 0.2440 - mae: 0.3362 - val_loss: 0.2849 - val_mae: 0.3615
Epoch 27/100
465/465  3s 6ms/step - loss: 0.2414 - mae: 0.3343 - val_loss: 0.2812 - val_mae: 0.3577
Epoch 28/100
465/465  5s 10ms/step - loss: 0.2413 - mae: 0.3345 - val_loss: 0.2770 - val_mae: 0.3564
Epoch 29/100
465/465  4s 9ms/step - loss: 0.2405 - mae: 0.3352 - val_loss: 0.2839 - val_mae: 0.3649
Epoch 30/100
465/465  3s 7ms/step - loss: 0.2383 - mae: 0.3328 - val_loss: 0.2860 - val_mae: 0.3560
Epoch 31/100
465/465  3s 6ms/step - loss: 0.2363 - mae: 0.3319 - val_loss: 0.2983 - val_mae: 0.3906
Epoch 32/100
465/465  3s 7ms/step - loss: 0.2398 - mae: 0.3320 - val_loss: 0.2839 - val_mae: 0.3680
Epoch 33/100
465/465  3s 7ms/step - loss: 0.2337 - mae: 0.3311 - val_loss: 0.2849 - val_mae: 0.3675
Epoch 34/100
465/465  3s 7ms/step - loss: 0.2328 - mae: 0.3287 - val_loss: 0.2729 - val_mae: 0.3581
Epoch 35/100
465/465  3s 6ms/step - loss: 0.2306 - mae: 0.3266 - val_loss: 0.2802 - val_mae: 0.3652
Epoch 36/100
465/465  3s 6ms/step - loss: 0.2304 - mae: 0.3277 - val_loss: 0.2786 - val_mae: 0.3585

Epoch 37/100

465/465  **6s** 7ms/step - loss: 0.2278 - mae: 0.3251 - val_loss: 0.2798 - val_mae: 0.3700

Epoch 38/100

465/465  **3s** 7ms/step - loss: 0.2258 - mae: 0.3241 - val_loss: 0.2856 - val_mae: 0.3805

Epoch 39/100

465/465  **4s** 7ms/step - loss: 0.2244 - mae: 0.3243 - val_loss: 0.2855 - val_mae: 0.3674

Epoch 40/100

465/465  **3s** 7ms/step - loss: 0.2232 - mae: 0.3226 - val_loss: 0.2781 - val_mae: 0.3670

Epoch 41/100

465/465  **3s** 7ms/step - loss: 0.2291 - mae: 0.3222 - val_loss: 0.2809 - val_mae: 0.3641

Epoch 42/100

465/465  **4s** 8ms/step - loss: 0.2216 - mae: 0.3223 - val_loss: 0.2632 - val_mae: 0.3551

Epoch 43/100

465/465  **3s** 7ms/step - loss: 0.2177 - mae: 0.3174 - val_loss: 0.2914 - val_mae: 0.3831

Epoch 44/100

465/465  **3s** 6ms/step - loss: 0.2168 - mae: 0.3173 - val_loss: 0.2645 - val_mae: 0.3501

Epoch 45/100

465/465  **3s** 6ms/step - loss: 0.2176 - mae: 0.3176 - val_loss: 0.2844 - val_mae: 0.3760

Epoch 46/100

465/465  **3s** 7ms/step - loss: 0.2133 - mae: 0.3165 - val_loss: 0.2710 - val_mae: 0.3540

Epoch 47/100

465/465  **3s** 7ms/step - loss: 0.2132 - mae: 0.3152 - val_loss: 0.2679 - val_mae: 0.3520

Epoch 48/100

465/465  **3s** 7ms/step - loss: 0.2110 - mae: 0.3136 - val_loss: 0.2852 - val_mae: 0.3699

Epoch 49/100

465/465  **3s** 6ms/step - loss: 0.2097 - mae: 0.3138 - val_loss: 0.2796 - val_mae: 0.3582

Epoch 50/100

465/465  **3s** 6ms/step - loss: 0.2108 - mae: 0.3124 - val_loss: 0.2777 - val_mae: 0.3600

Epoch 51/100

465/465  **3s** 7ms/step - loss: 0.2239 - mae: 0.3147 - val_loss: 0.2771 - val_mae: 0.3590

Epoch 52/100

465/465  **3s** 6ms/step - loss: 0.2085 - mae: 0.3126 - val_loss: 0.2769 - val_mae: 0.3572

Epoch 53/100

465/465  **3s** 6ms/step - loss: 0.2068 - mae: 0.3115 - val_loss: 0.2753 - val_mae: 0.3644

Epoch 54/100

465/465  **3s** 6ms/step - loss: 0.2035 - mae: 0.3086 - val_loss: 0.2739 - val_mae: 0.3506

Epoch 55/100

465/465  **3s** 6ms/step - loss: 0.2057 - mae: 0.3108 - val_loss: 0.2693 - val_mae: 0.3486

Epoch 56/100

465/465  **6s** 7ms/step - loss: 0.1999 - mae: 0.3050 - val_loss: 0.2717 - val_mae: 0.3575

Epoch 57/100

465/465  **3s** 6ms/step - loss: 0.2027 - mae: 0.3077 - val_loss: 0.3241 - val_mae: 0.4095

Epoch 58/100

465/465  **3s** 6ms/step - loss: 0.1999 - mae: 0.3070 - val_loss: 0.2673 - val_mae: 0.3512

Epoch 59/100

465/465  **3s** 6ms/step - loss: 0.1965 - mae: 0.3033 - val_loss: 0.2640 - val_mae: 0.3431

Epoch 60/100

465/465  **3s** 6ms/step - loss: 0.1967 - mae: 0.3026 - val_loss: 0.2908 - val_mae: 0.3740

Epoch 61/100

465/465  **3s** 6ms/step - loss: 0.1969 - mae: 0.3035 - val_loss: 0.2782 - val_mae: 0.3550

Epoch 62/100

465/465  **3s** 6ms/step - loss: 0.1964 - mae: 0.3031 - val_loss: 0.2743 - val_mae: 0.3476

Epoch 63/100

465/465  **3s** 6ms/step - loss: 0.1952 - mae: 0.3036 - val_loss: 0.2814 - val_mae: 0.3539

Epoch 64/100

465/465  **4s** 7ms/step - loss: 0.1966 - mae: 0.3045 - val_loss: 0.2822 - val_mae: 0.3636

Epoch 65/100

465/465  **3s** 6ms/step - loss: 0.1927 - mae: 0.3004 - val_loss: 0.2711 - val_mae: 0.3462

Epoch 66/100

465/465  **3s** 6ms/step - loss: 0.1940 - mae: 0.3021 - val_loss: 0.2667 - val_mae: 0.3471

Epoch 67/100

465/465  **3s** 6ms/step - loss: 0.1894 - mae: 0.2975 - val_loss: 0.2676 - val_mae: 0.3510

Epoch 68/100

465/465  **3s** 7ms/step - loss: 0.1898 - mae: 0.2996 - val_loss: 0.2636 - val_mae: 0.3453

Epoch 69/100

465/465  **3s** 6ms/step - loss: 0.1874 - mae: 0.2969 - val_loss: 0.2810 - val_mae: 0.3533

Epoch 70/100

465/465  **3s** 5ms/step - loss: 0.1866 - mae: 0.2973 - val_loss: 0.2876 - val_mae: 0.3742

Epoch 71/100

465/465  **3s** 6ms/step - loss: 0.1885 - mae: 0.2983 - val_loss: 0.2754 - val_mae: 0.3564

Epoch 72/100

465/465  **3s** 6ms/step - loss: 0.1843 - mae: 0.2953 - val_loss: 0.2800 - val_mae: 0.3589

Epoch 73/100

465/465  **3s** 6ms/step - loss: 0.1856 - mae: 0.2968 - val_loss: 0.2624 - val_mae: 0.3431

Epoch 74/100

465/465  **4s** 8ms/step - loss: 0.1836 - mae: 0.2931 - val_loss: 0.2745 - val_mae: 0.3539

Epoch 75/100

465/465  **3s** 6ms/step - loss: 0.1816 - mae: 0.2935 - val_loss: 0.2765 - val_mae: 0.3503

Epoch 76/100

465/465  **3s** 6ms/step - loss: 0.1837 - mae: 0.2946 - val_loss: 0.2777 - val_mae: 0.3648

Epoch 77/100

465/465  **3s** 6ms/step - loss: 0.1799 - mae: 0.2923 - val_loss: 0.2801 - val_mae: 0.3574

Epoch 78/100

465/465  **6s** 7ms/step - loss: 0.1794 - mae: 0.2921 - val_loss: 0.2689 - val_mae: 0.3524

Epoch 79/100

465/465  **3s** 6ms/step - loss: 0.1778 - mae: 0.2911 - val_loss: 0.2830 - val_mae: 0.3546

Epoch 80/100

465/465  **3s** 6ms/step - loss: 0.1765 - mae: 0.2906 - val_loss: 0.2680 - val_mae: 0.3542

Epoch 81/100

465/465  **3s** 6ms/step - loss: 0.1791 - mae: 0.2937 - val_loss: 0.2896 - val_mae: 0.3645

Epoch 82/100

465/465  **3s** 6ms/step - loss: 0.1808 - mae: 0.2921 - val_loss: 0.2740 - val_mae: 0.3518

Epoch 83/100

465/465  **3s** 6ms/step - loss: 0.1757 - mae: 0.2896 - val_loss: 0.2755 - val_mae: 0.3499

Epoch 84/100

465/465  **3s** 6ms/step - loss: 0.1748 - mae: 0.2887 - val_loss: 0.2685 - val_mae: 0.3464

Epoch 85/100

465/465  **3s** 6ms/step - loss: 0.1735 - mae: 0.2874 - val_loss: 0.2782 - val_mae: 0.3520

Epoch 86/100

465/465  **3s** 6ms/step - loss: 0.1744 - mae: 0.2878 - val_loss: 0.2740 - val_mae: 0.3512

Epoch 87/100

465/465  **3s** 6ms/step - loss: 0.1694 - mae: 0.2844 - val_loss: 0.2833 - val_mae: 0.3587

Epoch 88/100

465/465  **3s** 6ms/step - loss: 0.1710 - mae: 0.2859 - val_loss: 0.2682 - val_mae: 0.3510

Epoch 89/100

465/465  **3s** 6ms/step - loss: 0.1695 - mae: 0.2849 - val_loss: 0.2924 - val_mae: 0.3640

Epoch 90/100

465/465  **3s** 6ms/step - loss: 0.1803 - mae: 0.2921 - val_loss: 0.2762 - val_mae: 0.3505

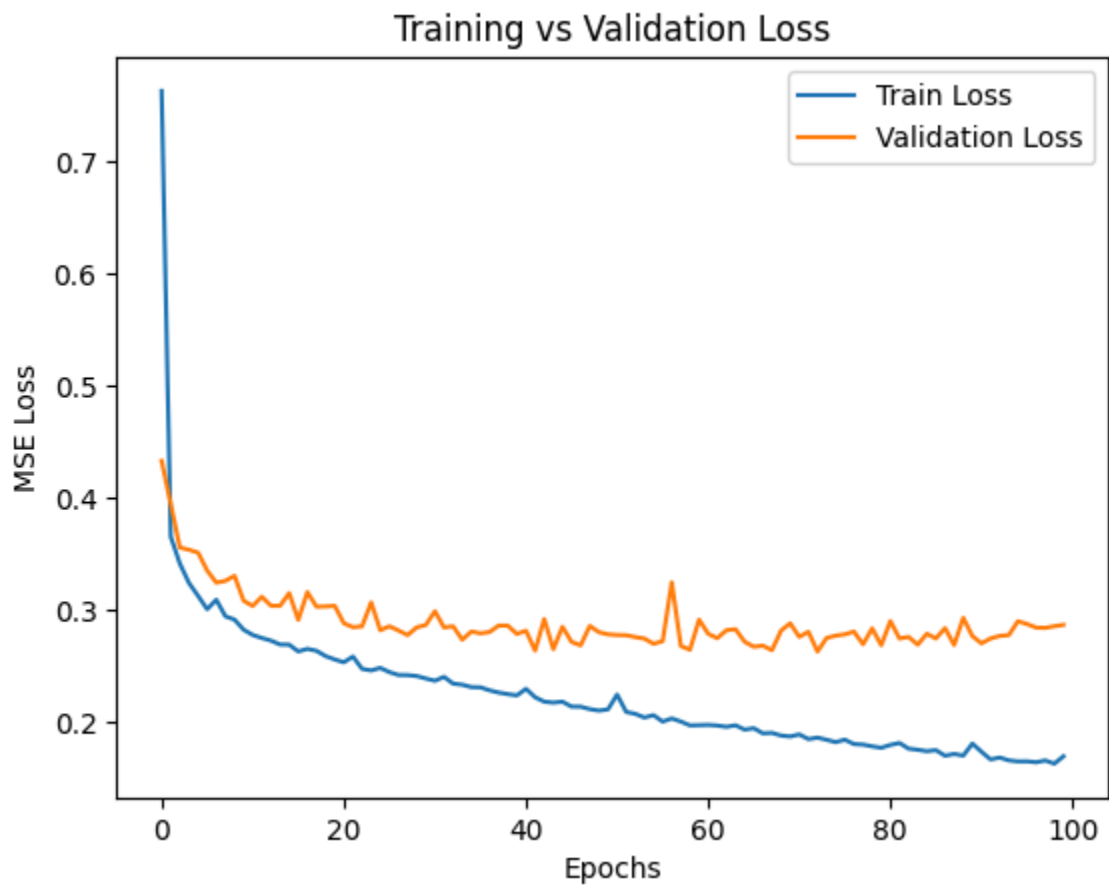
Epoch 91/100
465/465  **5s** 6ms/step - loss: 0.1730 - mae: 0.2875 - val_loss: 0.2696 - val_mae: 0.3510
Epoch 92/100
465/465  **3s** 6ms/step - loss: 0.1661 - mae: 0.2826 - val_loss: 0.2740 - val_mae: 0.3597
Epoch 93/100
465/465  **3s** 5ms/step - loss: 0.1678 - mae: 0.2840 - val_loss: 0.2763 - val_mae: 0.3592
Epoch 94/100
465/465  **3s** 6ms/step - loss: 0.1653 - mae: 0.2825 - val_loss: 0.2770 - val_mae: 0.3551
Epoch 95/100
465/465  **3s** 5ms/step - loss: 0.1644 - mae: 0.2810 - val_loss: 0.2894 - val_mae: 0.3564
Epoch 96/100
465/465  **3s** 6ms/step - loss: 0.1644 - mae: 0.2822 - val_loss: 0.2869 - val_mae: 0.3579
Epoch 97/100
465/465  **2s** 5ms/step - loss: 0.1636 - mae: 0.2805 - val_loss: 0.2837 - val_mae: 0.3555
Epoch 98/100
465/465  **3s** 6ms/step - loss: 0.1652 - mae: 0.2825 - val_loss: 0.2835 - val_mae: 0.3527
Epoch 99/100
465/465  **3s** 6ms/step - loss: 0.1624 - mae: 0.2782 - val_loss: 0.2849 - val_mae: 0.3647
Epoch 100/100
465/465  **3s** 6ms/step - loss: 0.1690 - mae: 0.2811 - val_loss: 0.2862 - val_mae: 0.3693
129/129  **1s** 5ms/step - loss: 0.2875 - mae: 0.3591
129/129  **1s** 4ms/step

Test MAE: 0.3591201603412628
RMSE: 0.5361918084920895
R2 Score: 0.7806015829653721
Actual vs Predicted

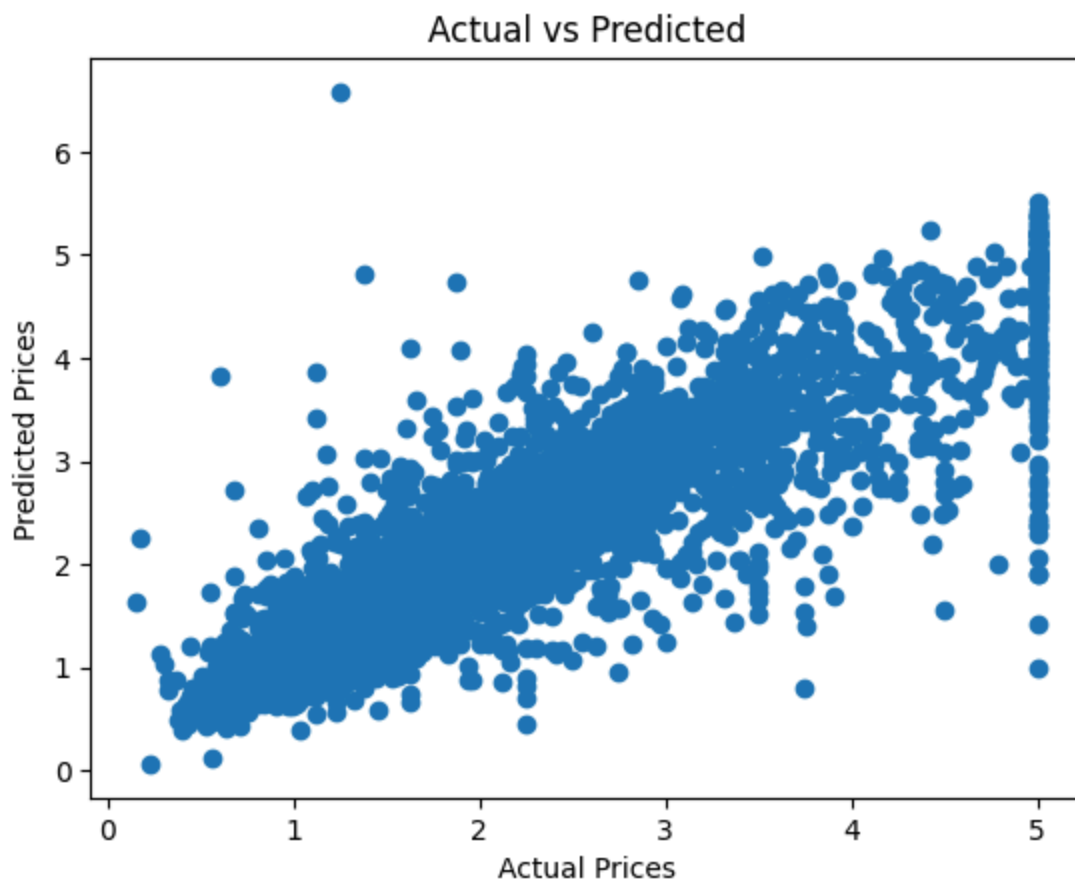
```
In [8]: # =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
```

```
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
```



Out[8]: Text(0.5, 1.0, 'Actual vs Predicted')



```
In [9]: print(history.history.keys())  
  
dict_keys(['loss', 'mae', 'val_loss', 'val_mae'])
```

```
In [10]: # for batch 64  
history_64 = model.fit(  
    X_train, y_train,  
    validation_split=0.1,  
    epochs=100,  
    batch_size=64,  
    verbose=0  
)
```

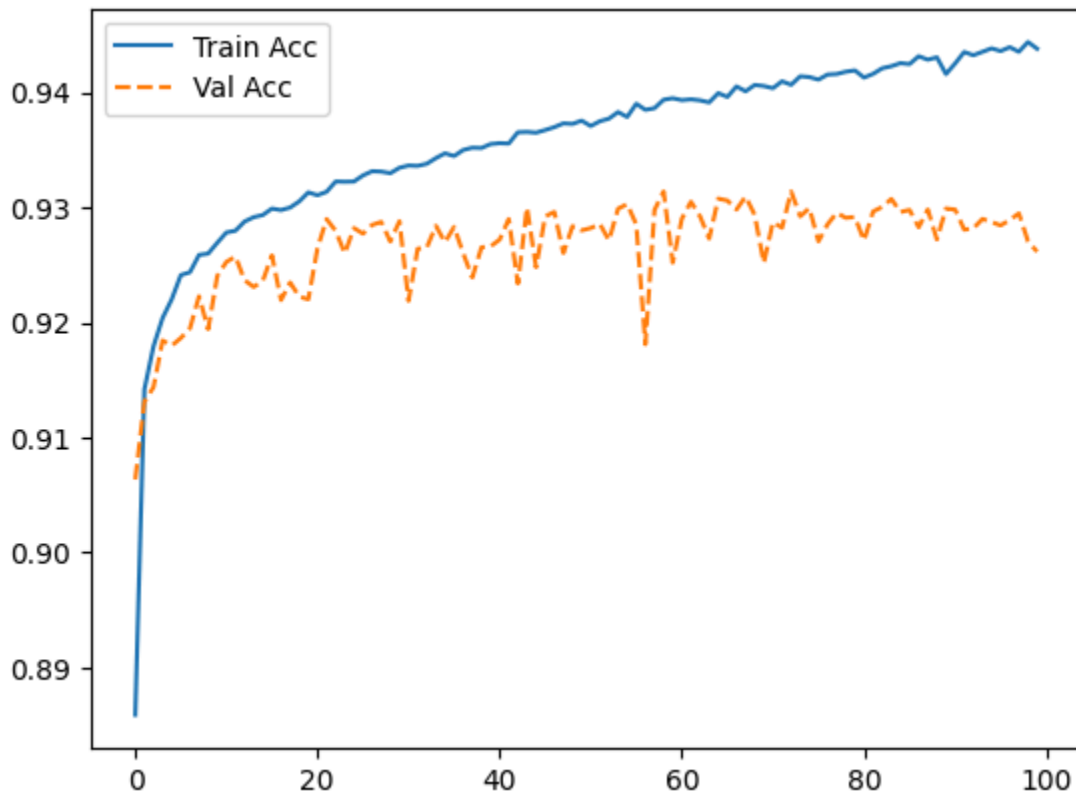
```
In [11]: test_loss_64, test_acc_64 = model.evaluate(X_test, y_test, verbose=0)
```

```
In [12]: # for batch 128  
history_128 = model.fit(  
    X_train, y_train,  
    validation_split=0.1,  
    epochs=100,  
    batch_size=128,  
    verbose=0  
)
```

```
In [13]: test_loss_128, test_acc_128 = model.evaluate(X_test, y_test, verbose=0)
```

```
In [14]: print(history.history.keys())  
  
dict_keys(['loss', 'mae', 'val_loss', 'val_mae'])
```

```
In [15]: max_val = np.max(y_train)  
  
train_acc_like = 1 - (np.array(history.history['mae']) / max_val)  
val_acc_like = 1 - (np.array(history.history['val_mae']) / max_val)  
  
plt.plot(train_acc_like, '-', label='Train Acc')  
plt.plot(val_acc_like, '--', label='Val Acc')  
plt.legend()  
plt.show()
```



```
In [ ]: import numpy as np  
import matplotlib.pyplot as plt  
  
max_val = np.max(y_train)  
  
# Convert MAE to accuracy-like  
train_32 = 1 - (np.array(history.history['mae']) / max_val)  
val_32 = 1 - (np.array(history.history['val_mae']) / max_val)  
  
train_64 = 1 - (np.array(history_64.history['mae']) / max_val)  
val_64 = 1 - (np.array(history_64.history['val_mae']) / max_val)  
  
train_128 = 1 - (np.array(history_128.history['mae']) / max_val)  
val_128 = 1 - (np.array(history_128.history['val_mae']) / max_val)
```

```

plt.figure(figsize=(12,7))

# Batch 32
plt.plot(train_32, '-', linewidth=2, label='Train (32)')
plt.plot(val_32, '--', linewidth=2, label='Val (32)')

# Batch 64
plt.plot(train_64, '-', linewidth=2, label='Train (64)')
plt.plot(val_64, '--', linewidth=2, label='Val (64)')

# Batch 128
plt.plot(train_128, '-', linewidth=2, label='Train (128)')
plt.plot(val_128, '--', linewidth=2, label='Val (128)')

plt.title("Accuracy Curve", fontsize=14)
plt.xlabel("Epochs", fontsize=12)
plt.ylabel("Accuracy Score", fontsize=12)

plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

